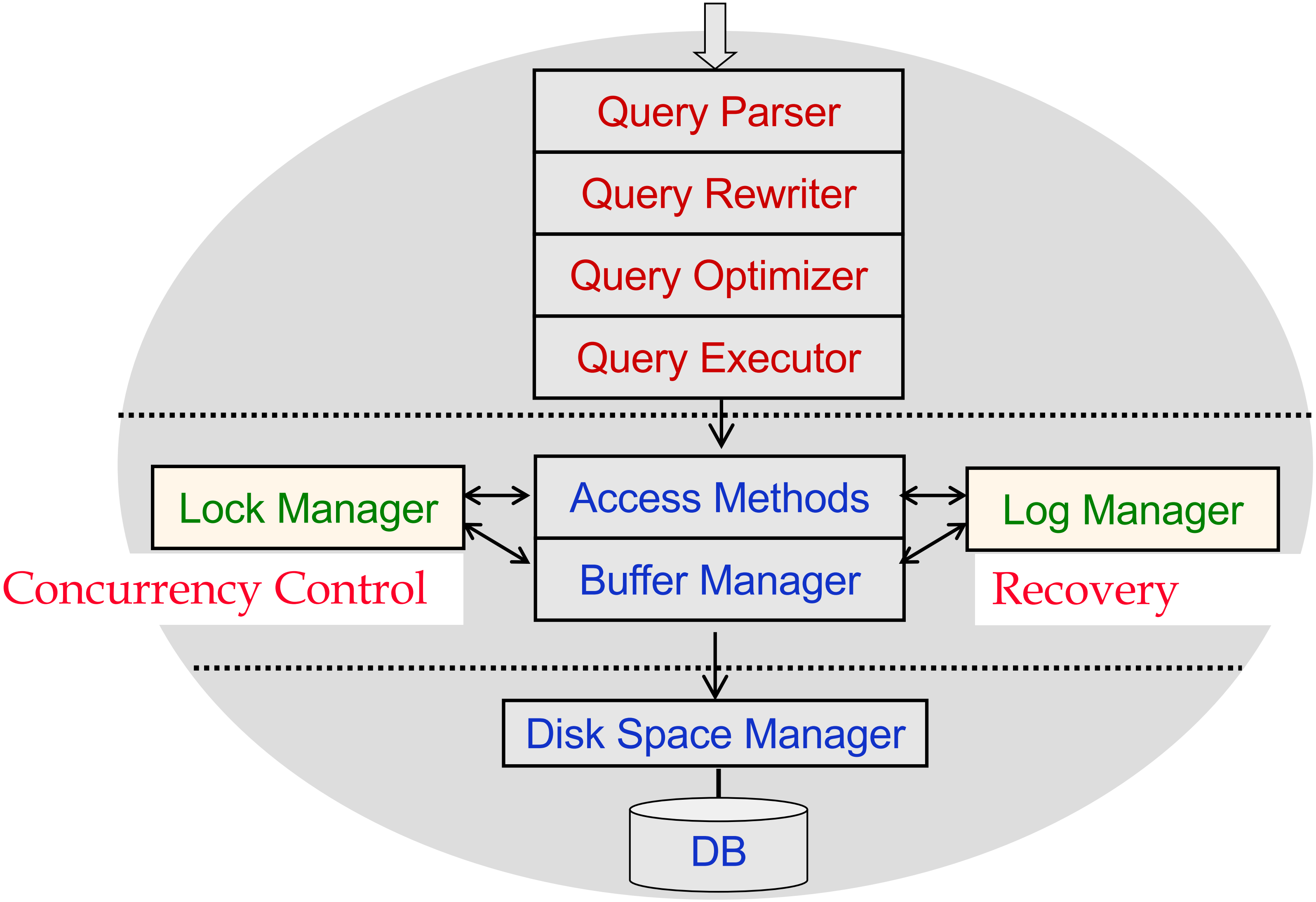
Transaction Management: Concurrency Control

Ioana Manolescu

Inria Saclay and Institut Polytechnique de Paris

Slides Courtesy of R. Ramakrishnan, J. Gehrke and Y. Diao

DBMS Architecture



Query Parser

Query Rewriter

Query Optimizer

Query Executor

Lock Manager

Access Methods

Log Manager

Buffer Manager

Concurrency Control

Recovery

Disk Space Manager

DB

Outline

- Transaction management overview
- Serializability & recoverability
- Lock-based concurrency control
- Optimistic concurrency control

Concurrent User Programs

- *Concurrent execution of user programs:* good for performance.
  - When task 1 is doing I/O, run task 2 to utilize the CPU.
  - Improve *average response time* (average delay that a user task experiences)
  - Improve *system throughput* (number of user tasks processed in each time unit)

Transactions

- User programs may do many things on the data retrieved.
  - E.g., operations on Bob's bank account.
  - E.g. transfer of money from account A to account B.
  - E.g., program searches for plane tickets, user looks at the tickets, … user buys the tickets.
- But the DBMS is only concerned about *what data is read from/written to the database*.
- A *transaction* (*xact*) is DBMS's abstract view of a run of a user program, simply, *a sequence of reads and writes*.

Concurrency

- Many users submit xacts, but each user thinks of his as executing by itself.
  - DMBS *interleaves* reads and writes of xacts for concurrency.
- *Consistency*: each xact starts and ends with a consistent state (i.e., satisfying all integrity constraints).
  - E.g., if an IC states that all accounts must have a positive balance, no transaction can violate this rule.
- ***Isolation***: execution of one xact appears isolated from others.
  - Intermediate (temporarily inconsistent) states should not be observed, e.g., account A being debited but B not being credited.

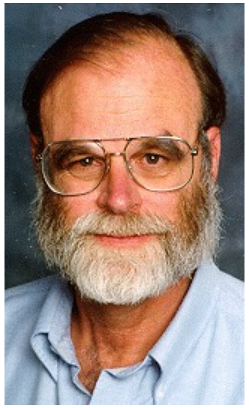
Recovery

- A transaction might *commit* after completing all its actions, or it could be *aborted* after executing some actions.
  - Constraint violation attempt; users change their mind; software or hardware failure
- ***Atomicity***: either all actions of a xact are performed or none of them is (*all-or-none*).
  - DBMS *logs* all actions so that it can *undo* the actions of aborted xacts.
- ***Durability***: once a user program has been notified of success, its effect will persist despite system failure.
  - DBMS *logs* all actions so that it can *redo* the actions of committed xacts.

"ACID" properties

Fundamental feature of data management systems:

- *Atomicity*: either all actions of a xact are performed or none of them is (*all-or-none*).
- *Consistency*: each xact starts and ends with a consistent state (i.e., satisfying all integrity constraints).
- *Isolation*: execution of one xact appears isolated from others.
- *Durability*: once a user program has been notified of success, its effect will persist despite system failure.



Jim Gray
Turing Award 1998
Transaction theory

In-course demo

Outline

- Transaction management overview
- Serializability & recoverability
- Lock-based concurrency control
- Optimistic concurrency control

Example

- Consider two transactions:

```
T1:    BEGIN   A=A+100,   B=B-100   END
T2:    BEGIN   A=1.06*A,   B=1.06*B   END
```

- 1st xact transfers $100 from B's account to A's.
- 2nd xact credits both accounts with a 6% interest payment.

- No guarantee that T1 will execute before T2 or vice-versa, if both are submitted together.

- However, the net effect must be *equivalent to* these two transactions running *serially* in some order!

Example (Contd.)

- Consider a possible interleaving *schedule*:

T1:	A=A+100,		B=B-100	
T2:		A=1.06*A,		B=1.06*B

- This has the same effect as the following serial schedule:

T1:	A=A+100, B=B-100	
T2:		A=1.06*A, B=1.06*B

  - A gets an interest after it receives 100 from B
  - B gets an interest after it gives 100 to A

Example (Contd.)

- Consider a possible interleaving *schedule*:

T1:	A=A+100,		B=B-100	
T2:		A=1.06*A,		B=1.06*B

- What about this schedule?

T1:	A=A+100,		B=B-100
T2:		A=1.06*A, B=1.06*B	

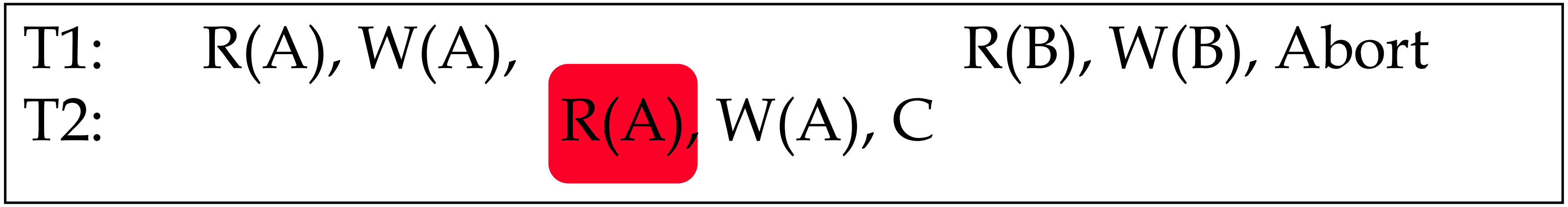
- A gets interest *after* gaining 100; B gets it *before* giving 100

The DBMS's view of this schedule:

T1:	R(A), W(A),		R(B), W(B)
T2:		R(A), W(A), R(B), W(B)	

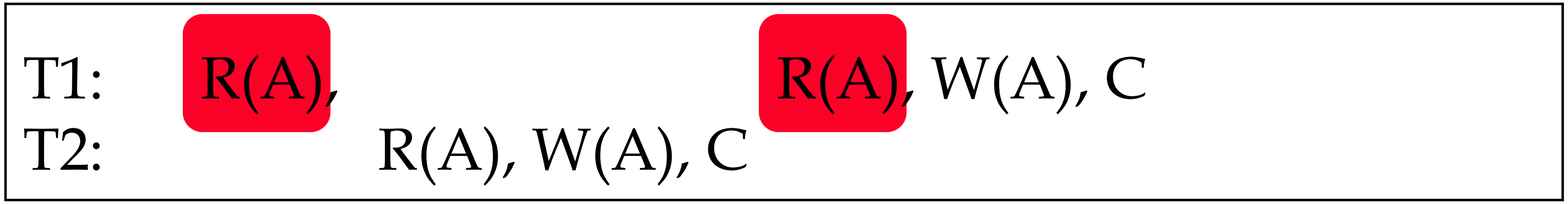
Anomalies with Interleaved Execution

- **Anomaly**: effect that no serial execution could yield
- Reading Uncommitted Data ("dirty reads"):



T1: R(A), W(A), R(B), W(B), Abort
T2: R(A), W(A), C

  - Similar to the previous example
- Unrepeatable Reads:



T1: R(A), R(A), W(A), C
T2: R(A), W(A), C

  - Example: A is the number of free tickets

Anomalies (Contd.)

- **Anomaly**: effect that no serial execution could yield
- Overwriting Uncommitted Data:

T1:	W(A),		W(B), C
T2:		W(A), W(B), C	

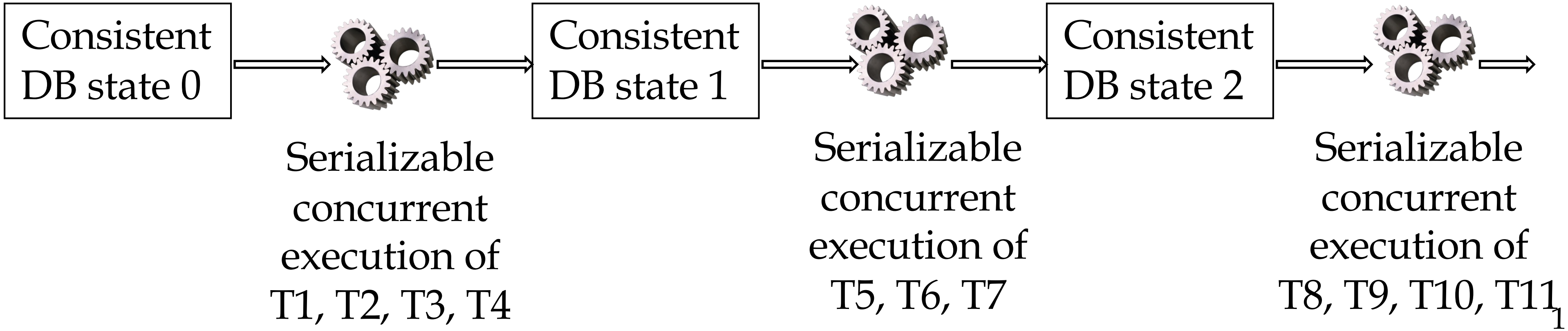
- Example
  - Alice and Bob are employees whose salaries must be kept equal.
  - T1 sets their salaries to $2000 and T2 sets the salaries to $1000.
  - What happens with the above schedule?
  - Intuition: "lost update" (the first on A)
- Note: this example features *blind writes* (not preceded by reads)

Scheduling Transactions

- *Serial schedule:* Schedule that **does not interleave** the actions of different transactions.

- *Equivalent schedules:* For any database state, the **effect** of executing the first schedule is **identical** to the effect of executing the second schedule.

- *Serializable schedule:* A schedule that is equivalent to some serial execution of the transactions.
  - **If each transaction preserves consistency, every serializable schedule preserves consistency.**



Consistent DB state 0

Serializable concurrent execution of T1, T2, T3, T4

Consistent DB state 1

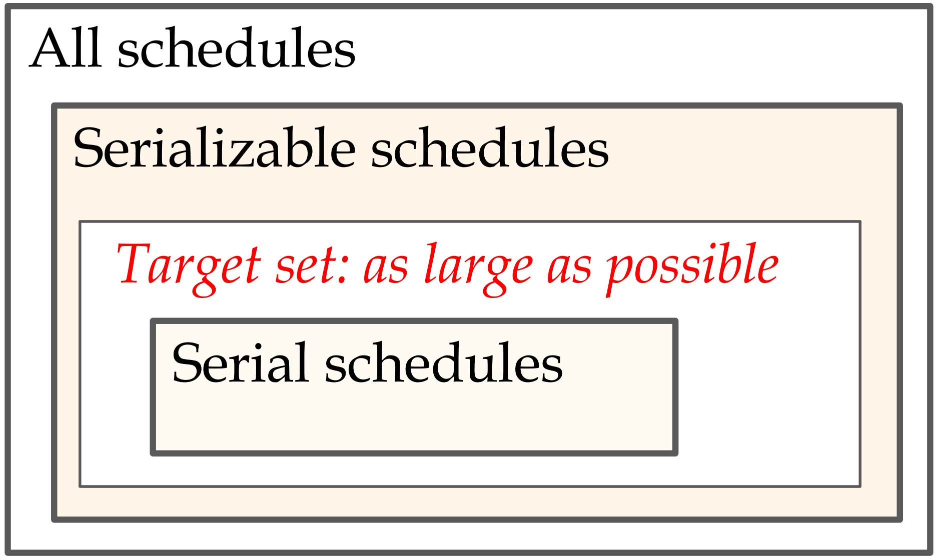
Serializable concurrent execution of T5, T6, T7

Consistent DB state 2

Serializable concurrent execution of T8, T9, T10, T11

Serializability

- *Serializability theory* concerns the schedules of transactions that are not (explicitly) aborted.
- Given a set of such xacts, ideally want to allow *any serializable schedule*.
  - Recognizing any serializable schedule is highly complex, if possible.
- Instead, allow only a *subset* of serializable schedules that are easy to detect.



All schedules

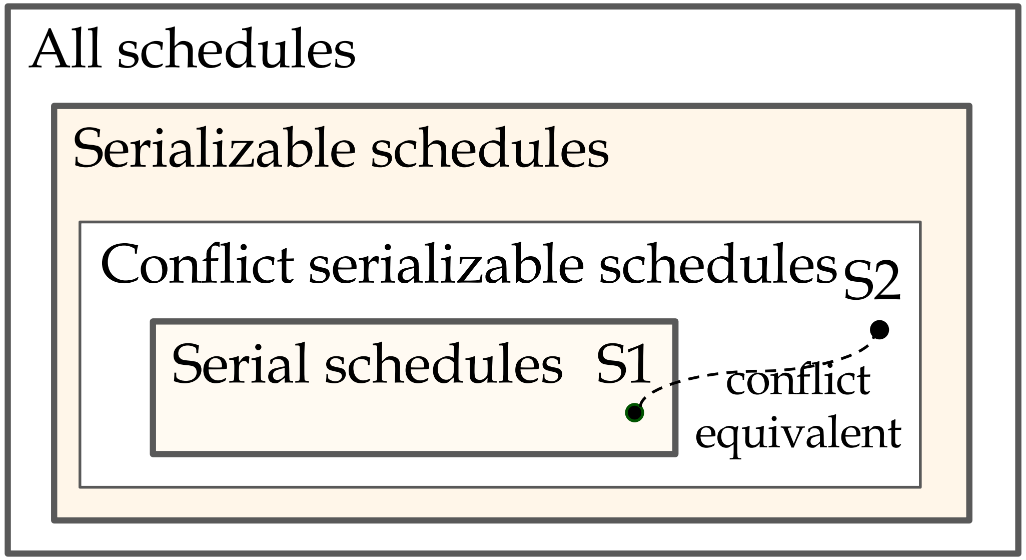
Serializable schedules

Target set: as large as possible

Serial schedules

Conflict Serializability

- **Conflicting actions** (or **conflicts**):
  - W(A), R(A) for the same data item A
  - W(A), W(A) for the same data item A
- Two schedules are *conflict equivalent* if:
  - They involve the same actions of the same transactions.
  - Every pair of *conflicting actions* is ordered the same way.
- S is *conflict serializable* if S is conflict equivalent to some serial schedule.



All schedules

Serializable schedules

Conflict serializable schedules

Serial schedules

S1

S2

conflict equivalent

How to Decide Conflict Serializability?

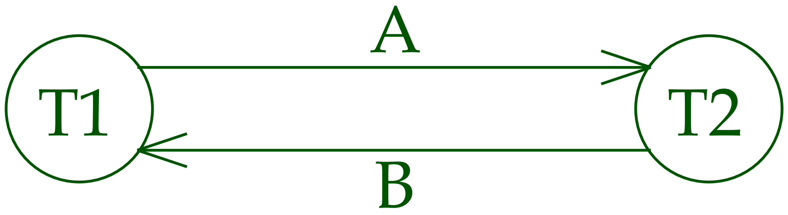
- Precedence (or dependency) graph of a given schedule:
  - One node per Xact;
  - Edge from Xact *Ti* to Xact *Tj* if an action of *Ti* *precedes in the schedule* and *conflicts with* one of *Tj*'s actions

Conflict on data object O?	**Read by Ti**	**Write by Ti**
Read by Tj		✗
Write by Tj	✗	✗

- **Theorem**: Schedule is conflict serializable *if and only if* its precedence graph is acyclic.

Example

T1:	R(A), W(A),		R(B), W(B)
T2:		R(A), W(A), R(B), W(B)	



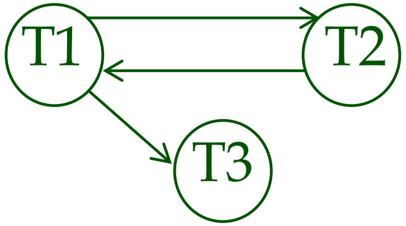
Precedence graph

- The schedule is not conflict serializable:
  - The output of T1 depends on T2, and vice-versa.

Weaker Condition on Serializability

- Conflict serializability is *sufficient, but not necessary* for serializability. Example:

S1: interleaved schedule



T1:	R(A)		W(A)	
T2:		W(A)		
T3:				W(A)

S2: serial schedule

T1:	R(A),W(A)		
T2:		W(A)	
T3:			W(A)

S1, S2 equivalent:

- Same final result since T3 overwrites T1 and T2.
- Same data seen by T1 before doing its write.
- T2 and T3 didn't read anything → previous writes don't matter.

View Serializability

- Schedules S1 and S2 are view equivalent if for any Ti, Tj:
  - If Ti *reads initial value of A* in S1, then Ti also reads initial value of A in S2
  - If Ti *reads value of A written by* Tj in S1, then Ti also reads value of A written by Tj in S2
  - If Ti *writes final value of A* in S1, then Ti also writes final value of A in S2

Previous conflicting schedule:

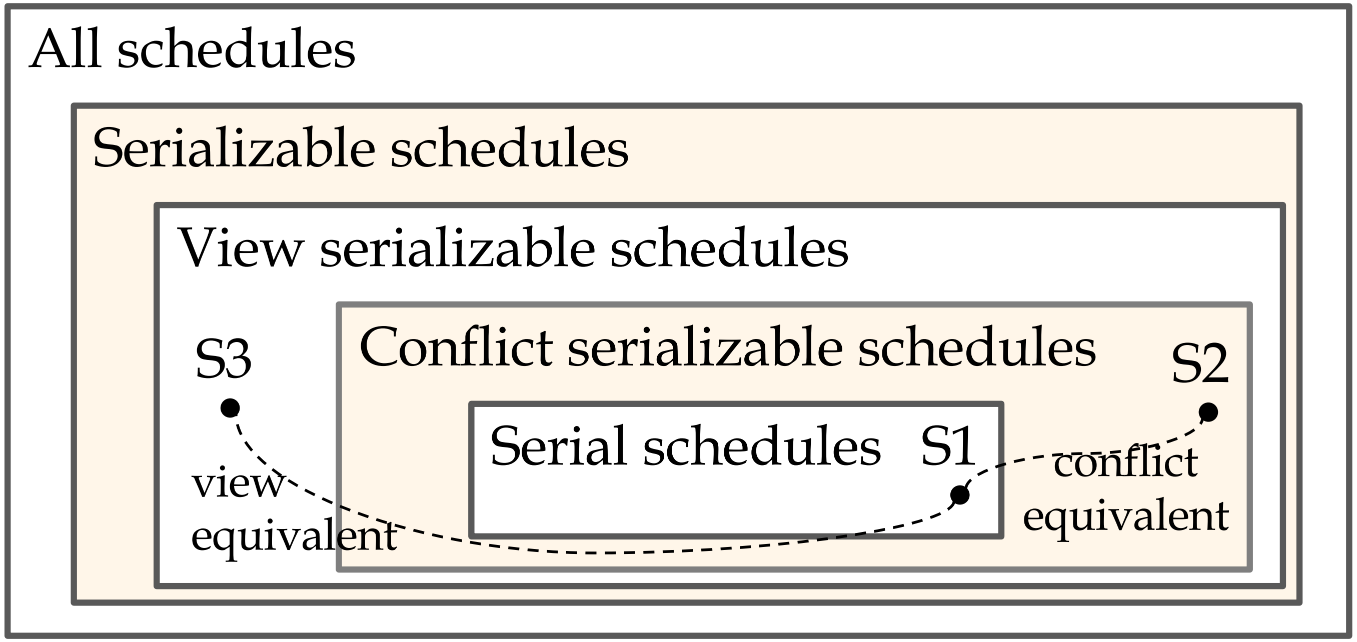
T1:	R(A)		W(A)	
T2:		W(A)		
T3:				W(A)

Previous serial schedule:

T1:	R(A),W(A)		
T2:		W(A)	
T3:			W(A)

View Serializability (Contd.)

- Definition: A schedule is *view serializable* if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is view serializable. The opposite does not hold.
- Every view serializable schedule that is not conflict serializable contains a *blind write*.


All schedules
Serializable schedules
View serializable schedules
Conflict serializable schedules
Serial schedules
S1
S2
S3
view
equivalent
conflict
equivalent

Extension to More Transactions

- So far we have defined serializability for schedules involving **committed** xacts.
- Extending to schedule S which involves also **aborted** xacts:
  - S is serializable iff its effects are identical to those of a serial schedule over the committed xacts of S
    - The effects of possible aborted xacts of S must be undone to preserve atomicity (A)
- Also, system failures may occur when an **ongoing** xact has neither aborted not committed (yet)
  - As we will see, recovery after failure starts by aborting all ongoing (yet-uncommitted) xact.
  - Thus, only focus on aborted and committed ones.

Recoverability

- *Recoverability theory* concerns schedules that involve *aborted* transactions: we still need to preserve ACID!

T1:	R(A),W(A)			Abort
T2:		R(A),W(A)	Commit	

  - For durability (D), the write of T2 needs to persist.
  - But T2 read the write of T1 which, for atomicity (A), should "not have left any trace in the system"!
  - No way to *return to a consistent state* after T1 aborts: it impacted T2, which cannot be undone after Commit
  - "Impossible to *recover*" from the failure of T1

Def: A schedule is *recoverable* iff for any Abort/Commit combination, it is possible to recover a consistent state

Thm (criterium): A schedule S is recoverable if each xact *commits* only after all xacts from which it read have committed.

Recoverability

- *Recoverability theory* concerns schedules that involve *aborted* transactions: we still need to preserve ACID!

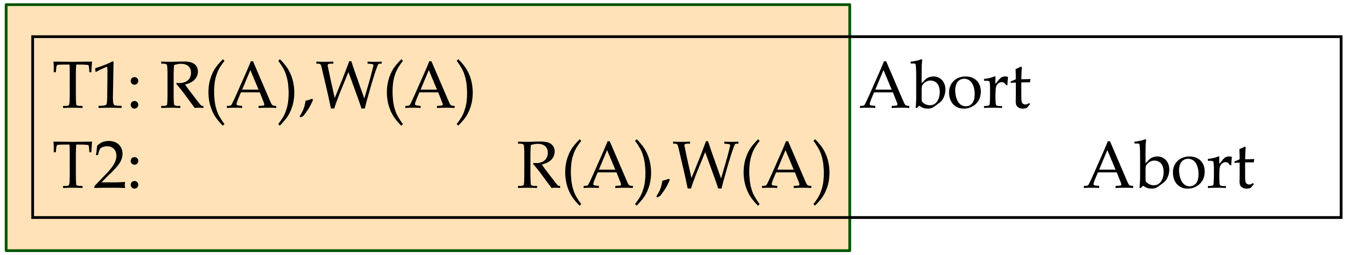
The DBMS fully controls when Xacts commit, read or write, because it determines the schedule.
The DBMS cannot fully control when transactions abort!
Aborts can be due to user decisions, Xacts trying to violate constraints, time-outs, system failure …

Ensuring recoverability means: *taking, at every step, scheduling decisions that guarantee it, regardless of which Xact Aborts when in the future.*

Def: A schedule is *recoverable* iff for any Abort/Commit combination, it is possible to recover a consistent state

Thm (criterium): A schedule S is recoverable if each xact *commits* only after all xacts from which it read have committed.

Recoverability (Contd.)



```
T1: R(A),W(A)                Abort
T2:             R(A),W(A)          Abort
```

Recoverable, but with cascading aborts.

- Aborts (also called *rollbacks* because the effects of the aborted transactions must be *rolled back*) may incur a high performance penalty → prefer *cascadeless* schedules
- **Def:** S avoids cascading rollback if no abort requires another abort in order to recover a consistent state
- **Thm (criterium)**: S *avoids cascading rollback* if each xact may *read* only those values written by committed xacts.

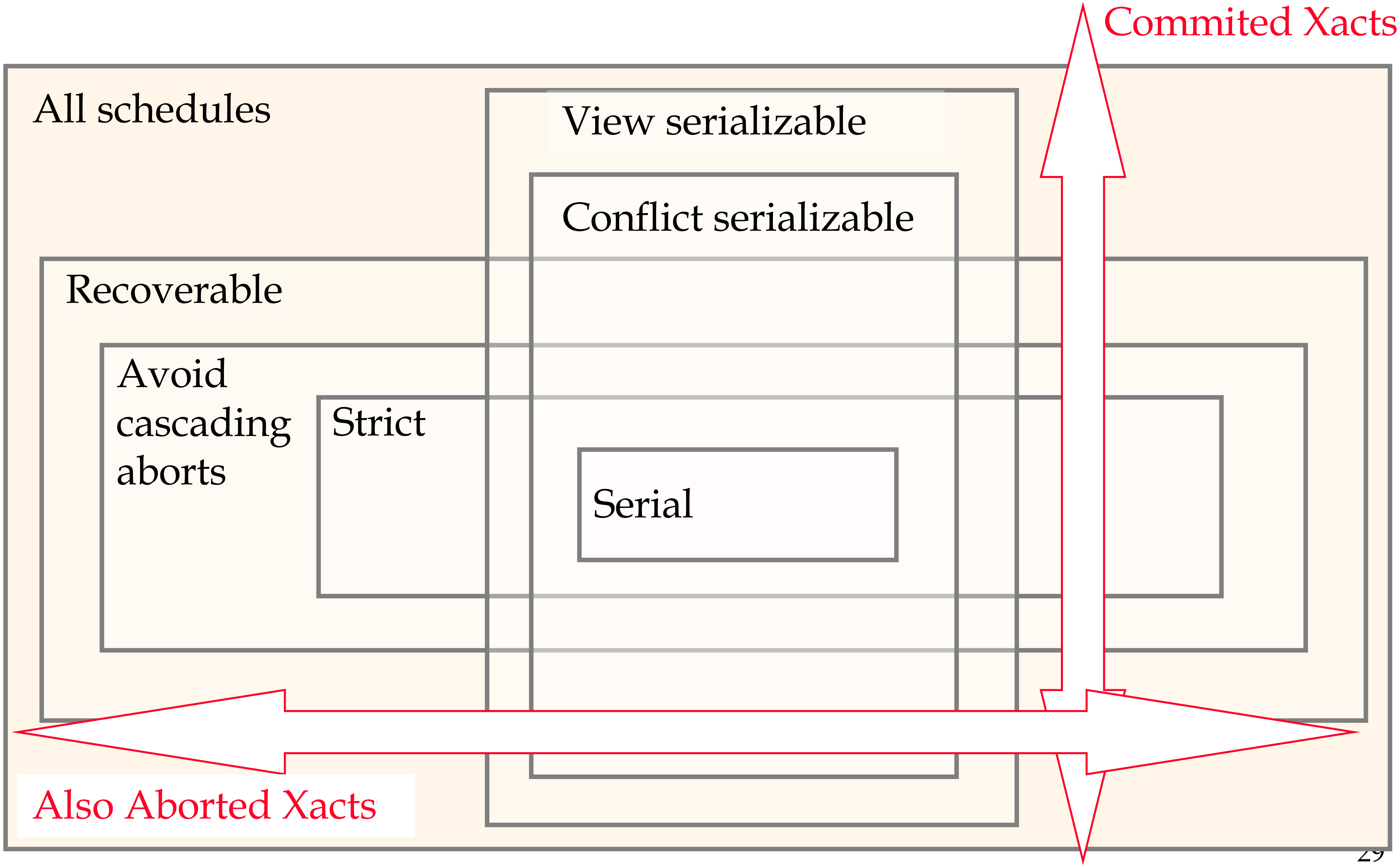
Recoverability (Contd.)

T1:		R(A), W(A)		Abort	
T2:	R(A)		W(A)		Commit

Recoverable, avoids cascading rollback.
Update of A by T2 may be lost when rolling back T1!
Need to consider other xacts during abort

- S is *strict* if each xact may *read or write* only those values written by committed xacts.
  - This excludes W(A) by T2 in this example.
  - Any strict schedule is also cascadeless.
  - In strict schedules, actions of aborted xacts can be simply undone by restoring the original values of modified objects.

Venn Diagram for Schedules



Commited Xacts

All schedules

View serializable

Conflict serializable

Recoverable

Avoid cascading aborts

Strict

Serial

Also Aborted Xacts

Outline

- Transaction management overview
- Serializability & recoverability
- Lock-based concurrency control
- Optimistic concurrency control

Lock-based Concurrency Control

- **Lock**: software artefact to control access to a resource
- Two basic kinds of locks: **Shared** (S) and **Exclusive** (X)
- Lock compatibility matrix:

Ti \ Tj	**S(A)**	**X(A)**
S(A)	✓	✗
X(A)	✗	✗

- Shared (resp. exclusive) lock also called a *read* (resp. *write*) *lock*
- Any xact **needs an S lock in order to read** and an **X lock in order to write**.
- If a xact needs a lock not currently available, it **queues** for that lock → locks implement (determine) schedule!

Locking Protocols

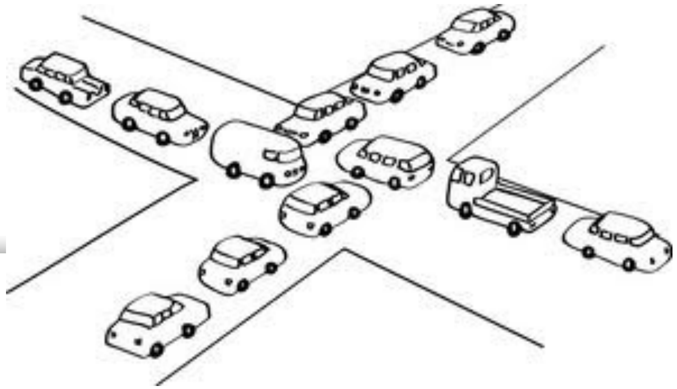
- Lock/unlock requests become part of the sequence of operations of a transaction
- A **locking protocol** is a rule governing when, during its execution time, a transaction can request and release locks

1. If concurrency control is ensured using *locks*
2. ... and xacts follow a specific *locking protocol*
3. ... then, the "go/wait" decisions imposed to xacts will lead to *schedulings* with specific *guaranteed properties*

We will see some important protocols next.

Trade-off: enable concurrency vs. ensure good properties

Deadlocks

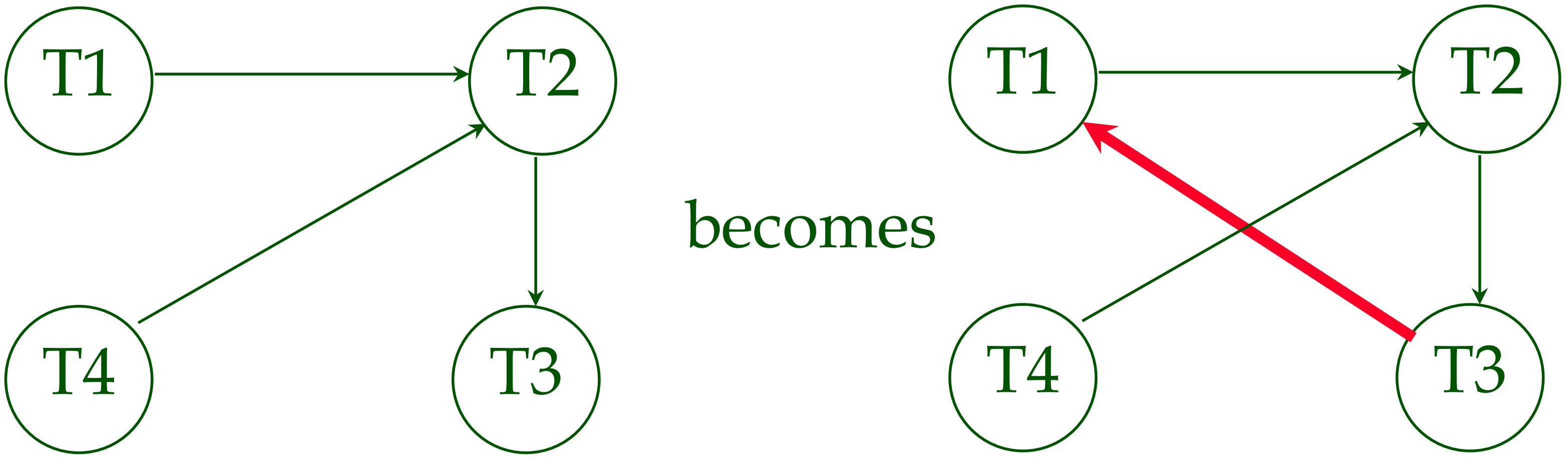


- Deadlock: Cycle of transactions waiting for locks to be released by each other.

- Two ways of dealing with deadlocks:
  - Deadlock *detection*
  - Deadlock *prevention*: try to reduce or eliminate the risk of deadlocks by specific locking protocols

Deadlock Detection (Contd.)

T1:	S(A), R(A),		S(B)				
T2:		X(B), W(B)			X(C)		
T3:				S(C), R(C)			X(A)
T4:						X(B)	

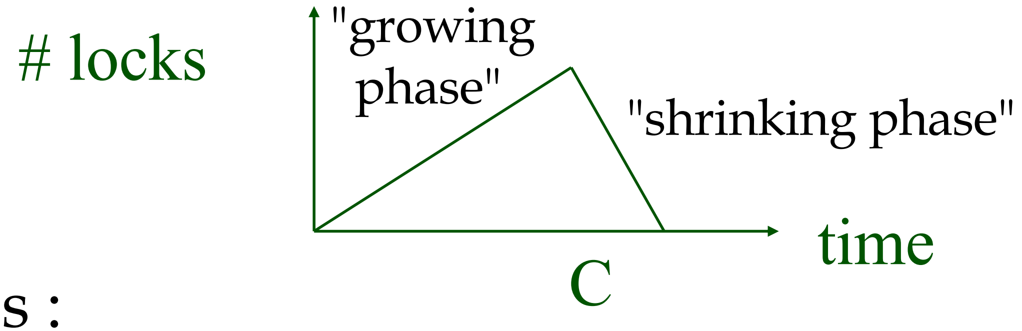

T1
T2
T4
T3
becomes
T1
T2
T4
T3

Deadlock Detection

- Create a waits-for graph:
  - Nodes are Xacts.
  - There is an edge from Xact Ti to Xact Tj if Ti is *waiting for* Tj to release a lock.
    - Ti wants a lock that Tj holds
  - Note the difference from the precedence graph for conflict serializability.
- Periodically **check for cycles**, indicating deadlocks, in the waits-for graph.
  - Resolve a deadlock by aborting a transaction on the cycle and releasing all its locks.

Two-Phase Locking (2PL) Protocol

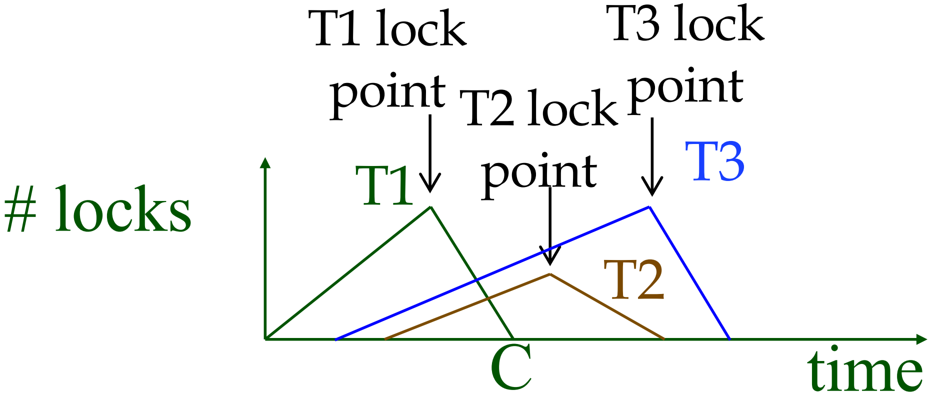
- Two-Phase Locking Protocol
  - *A transaction cannot request additional locks once it releases any locks.*



- Examples :
  - T1 reads R twice while T2 updates a few tuples.
  - T1 can start releasing locks during the 2nd pass of the scan → more concurrency (T2 can start in parallel)

2PL (continued)

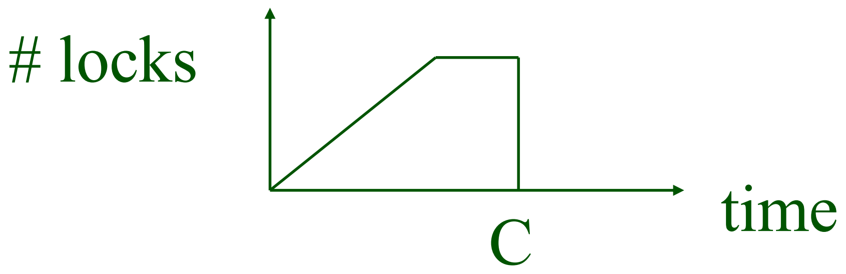
- **Theorem**: 2PL ensures acyclicity of precedence graph.
  - 2PL only allows *conflict serializable* schedules.
  - An equivalent serial schedule is given by the order of the moments (*lock points*) when xacts enter their shrinking phase



- 2PL may still lead to *deadlocks*
- 2PL is not strict (→ may require complex abort processing, in particular cascading rollback)

Strict 2PL

- Strict Two-Phase Locking (Strict 2PL) Protocol:
  - All locks held by a transaction are released **when the transaction completes**.
  - The number of locks held by any transaction evolves in time as follows:



Strict 2PL (Contd.)

- **Theorem**: Strict 2PL allows only schedules whose precedence graph is acyclic.
  - Strict 2PL only allows *conflict serializable schedules*!

- Strict 2PL is strict with respect to recoverability.
  - Strict 2PL is *recoverable without anomalies related to aborted transactions*.
  - Hence, it simplifies transaction aborts.
  - Strict 2PL is guaranteed to avoid cascading rollback

- Strict 2PL still allows deadlocks

Deadlock Prevention

- Assign **priorities** to xacts, based on **timestamps**.
  - The *older* the timestamp, the *higher* the xact's priority.

Two policies are possible. When Ti wants a lock that Tj holds:

- *Wait-Die*: If Ti has higher priority, Ti waits for Tj; otherwise Ti aborts.
  - *Lower priority xacts never wait.*
- *Wound-wait*: If Ti has higher priority, Tj aborts; otherwise Ti waits.
  - *Higher priority xacts never wait.*
- When a transaction aborts and then re-starts, make sure it has its original timestamp so its priority increases.

The Phantom Problem

- If we consider insertions and updates of records in a DB, even Strict 2PL won't assure *serializability*.
- Example:
  - T1 locks *all* pages containing sailor records with *rating* = 1, and finds oldest sailor among them, say, *age* = 71.
  - T2 next *inserts* a **new** sailor: *rating* = 1, *age* = 96.
  - T1 now reads the oldest sailor again; now, *age* = 96 appears as a *phantom*!
  - **Problem**: T1 assumes that it has locked *all* sailor records with *rating* = 1.
  - This is only true if nobody *adds/updates records while T1 is running*!

The Phantom Problem (Contd.)

- Phantoms can occur with both inserts and updates.
  - E.g., update a sailor's *rating* from 3 to 1.
- Fundamental reason:
  - We discussed a transaction model which reads, writes and locks *individual data items*.
  - In practice, transactions include queries that *dynamically* define sets of items based on *predicates*, e.g.:

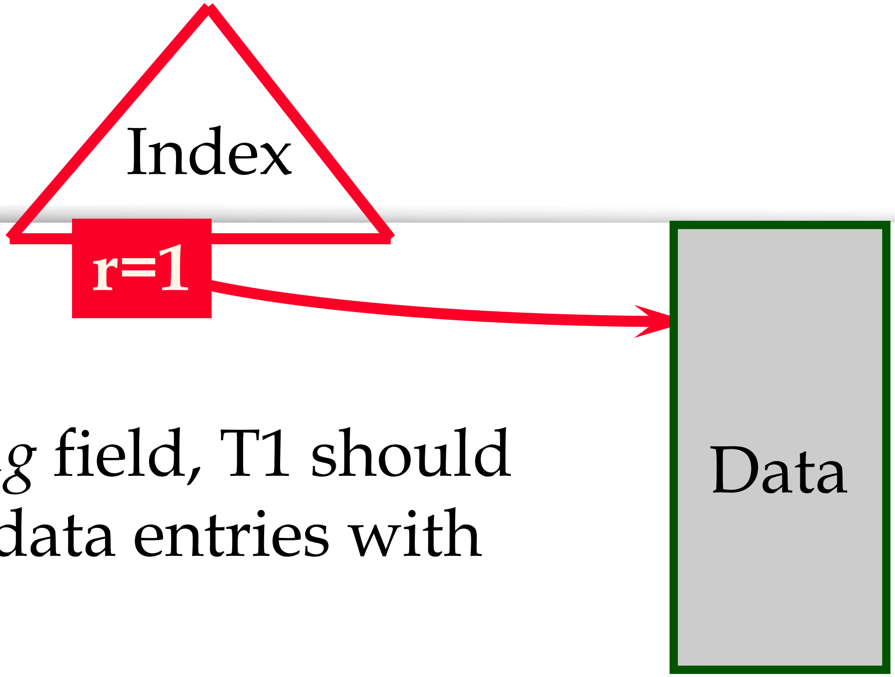
```
BEGIN TRANSACTION
SELECT max(age) FROM Sailors WHERE rating = 1; …
END TRANSACTION
```

  - When the query is executing, all the records satisfying this predicate at a particular time can be locked.
  - Locking individual items, however, cannot prevent later addition of a record satisfying this predicate.

The Phantom Problem (Contd.)

- Phantoms can occur with both inserts and updates.
  - E.g., update a sailor's *rating* from 3 to 1.
- Fundamental reason:
  - We discussed a transaction model which reads, writes and locks *individual* data items.
- Need a mechanism to enforce the assumption that T1 indeed holds locks on *all* records satisfying a condition.
  - Predicate locking:
    - e.g. Q1: *age* > 2**sal*, Q2: (*age* > 50 or *age* <30) and *sal* >10K
    - Requires checking predicate on each new or changed record
  - Index locking: used in practice

Index Locking



- If there is an index on the *rating* field, T1 should lock the *index page* containing data entries with *rating* = 1.
  - If there are no data entries with *rating* = 1, T1 must lock the index page where such a data entry *would* be, if it existed!

- If there is no suitable index, T1 must lock the entire file/table to prevent new pages from being added.
  - E.g., the new record with *rating* = 1, age=96

Lock Management

- *Lock manager* handles the lock and unlock requests
- *Transaction table*: xact id → <locks held in xact>
- *Lock table*: object id → lock table entry
  - Object can be a page, a record, etc.
  - Lock table entry:
    - *Number* of transaction(s) currently holding (a) lock(s)
    - *Type* of lock held (shared or exclusive)
    - *Queue* of lock requests (who is waiting for a lock on this object)

Lock and Unlock

- When a xact requests a lock on an object O:
  - If an S lock is requested, O is not currently locked in the X mode, and the queue of requests is empty, then…
  - If an X lock is requested, no xact currently holds a lock on O, then…
  - Otherwise, …

- When a xact aborts or commits:
  - it releases all its locks, and
  - when the lock on an object O' is released, its lock table entry is updated and the request at the head of the queue is processed.

Other Lock Management Issues

- Lock and unlock must be **atomic** operations.
  - Important when several instances of the lock manager code can execute concurrently.
  - Implemented using an OS synchronization mechanism such as a semaphore.

- ***Lock upgrade***: A xact T that holds a S lock can get "favor treatment" when asking for an X lock on same object O:
  - Grant X lock immediately if no other xact has a S lock
  - Otherwise, put T first in the waiting queue for this lock (even if other xacts already waited there)
    - No point in making T wait while holding S lock: risk of deadlock if another T' waits on a lock detained by T

Outline

- Transaction management overview
- Serializability & recoverability
- Lock-based concurrency control
- Optimistic concurrency control

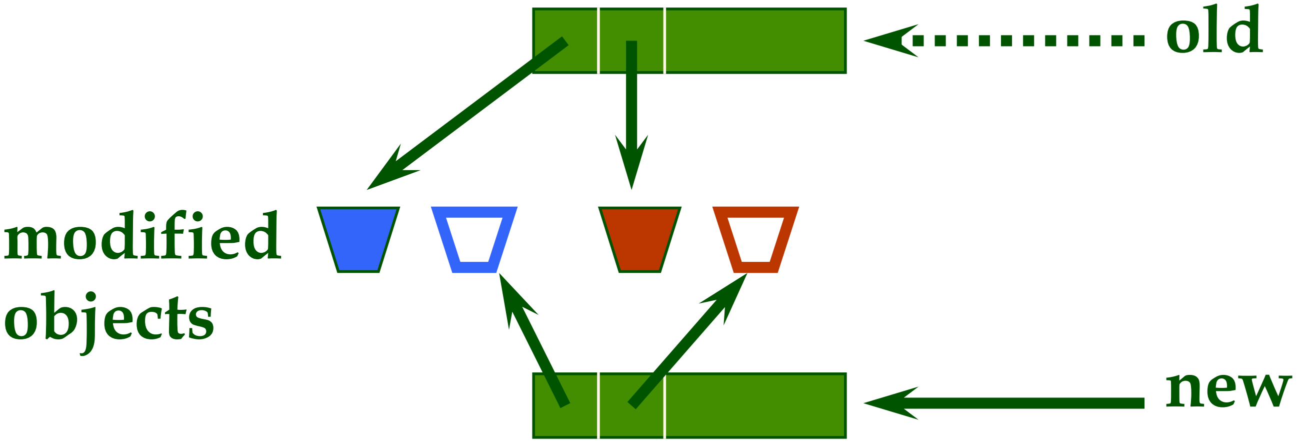
Optimistic CC (Kung-Robinson)

- Locking is a *conservative (pessimistic)* approach to preventing conflicts:
  - Lock management overhead.
  - Deadlock detection/resolution.
  - Lock contention for heavily used objects.
  - Phantom problem.

- Optimistic idea: If conflicts are rare, we gain concurrency if we do not lock, and instead *check for conflicts before xacts commit*.

Kung-Robinson Model

- Xacts have three phases:
  1. READ: Xacts read from the database, but make changes to *local* (*private*) *copies* of objects.
  2. VALIDATE: Check for conflicts. If there is a conflict, abort (clear the local copy & restart)!
  3. WRITE: Make local copies of changes public.



Validation

- Goal: guarantee that only serializable schedules result.
- Technique: find an equivalent serializable schedule
  - Assign to each xact an Xact id (*timestamp*) at the beginning of the *validation* phase,
  - *Check if the timestamp-ordering of xacts is an equivalent serial order.*
- Three test conditions (see next) ensure an equivalent serializable schedule. The conditions use:
  - ReadSet(Ti): objects read by Xact Ti
  - WriteSet(Ti): objects modified by Ti
- Any of these conditions ensures that Ti's modifications are not visible to Tj → validation succeeds (equiv. "Ti, then Tj")
- Otherwise, Tj aborts.

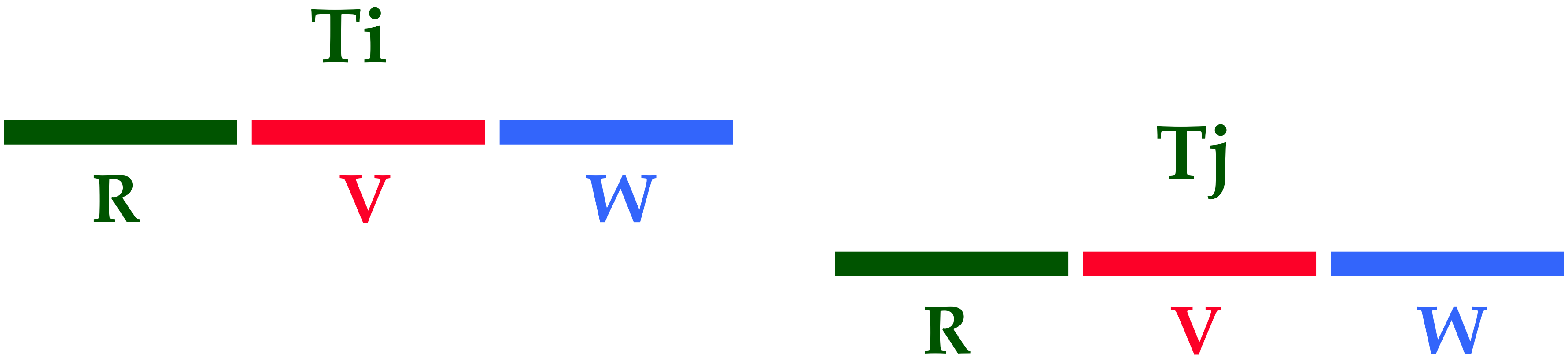
Overheads in Optimistic CC

- Record read/write activity in ReadSet/WriteSet per Xact.
  - Must create and destroy these sets as needed.

- Assignment of Xact id, validation, and the Write phase are inside a *critical section*
  - When a process is in a critical section, the system suspends all other processes; here, no concurrent critical section of another Xact.
  - If Write phase is long, major drawback.
- Optimization: no critical section for read-only xacts.
- Restart Xacts that fail validation.
  - Work done so far is wasted; requires clean-up.

When validating Tj, Test 1:

- For all i such that Ti < Tj, check that Ti completes before Tj begins.



Ti

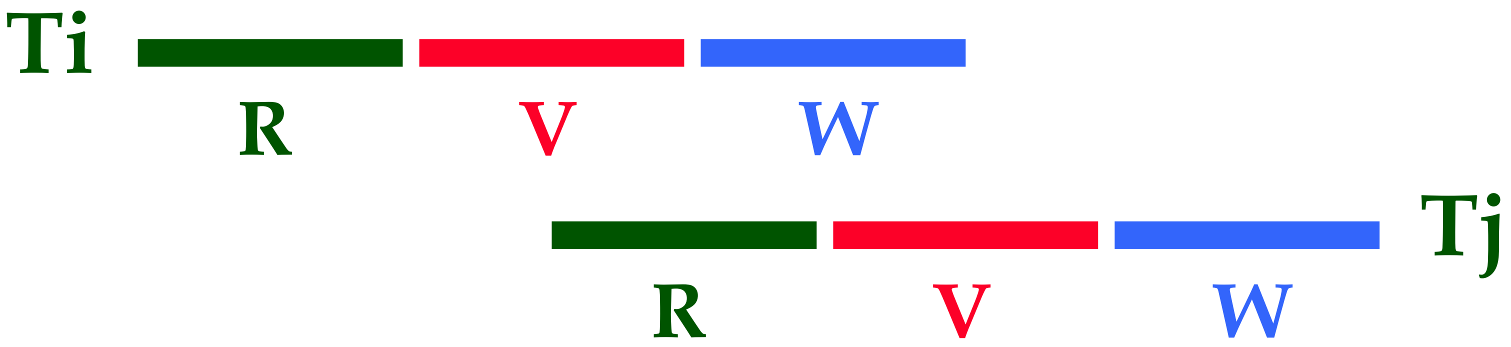
R **V** **W**

Tj

R **V** **W**

For those Ti failing Test 1, try Test 2…

- For all i such that Ti < Tj, check that:
  - Ti completes before Tj begins its Write phase, and
  - WriteSet(Ti) ∩ ReadSet(Tj) is empty.

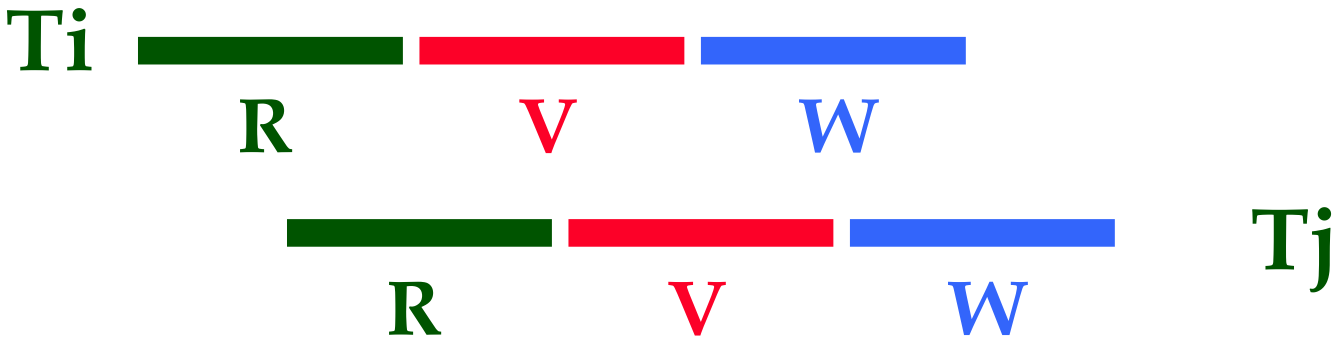


Can Tj read dirty data? Can Tj overwrite Ti's writes?

- Check correctness: all three types of conflicts (W-R, R-W, W-W), if present, go one way only.

For those Ti failing Test1 and Test2, try Test 3

- For all i such that Ti < Tj, check that:
    - Ti completes Read phase before Tj does, and
    - WriteSet(Ti) ∩ ReadSet(Tj) is empty, and
    - WriteSet(Ti) ∩ WriteSet(Tj) is empty.



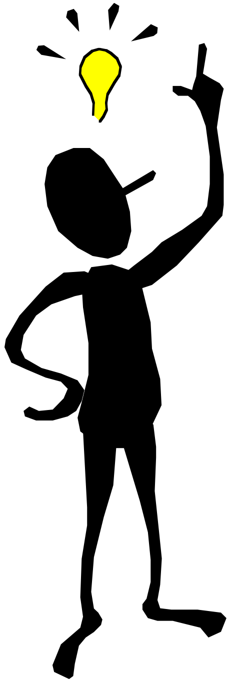
Does Tj read dirty data? Does Tj overwrite Ti's writes?

- Why is it correct?

Comments on Optimistic CC

- Compared to Locking
  - Optimistic CC: assumes no conflicts first, only fixes problems when conflicts appear, by *restarting* xacts.
  - Locking (pessimistic): conflicts are prevented in advance, by *blocking* from (potentially) nonserializable actions.
- Works well for some workloads:
  - All xacts are readers.
  - *Low interference*, e.g. large amount of data, each xact accessing a small (likely non-overlapping) amount of data.
- Deadlock free, but may have starvation.
- No phantom problem!

Questions

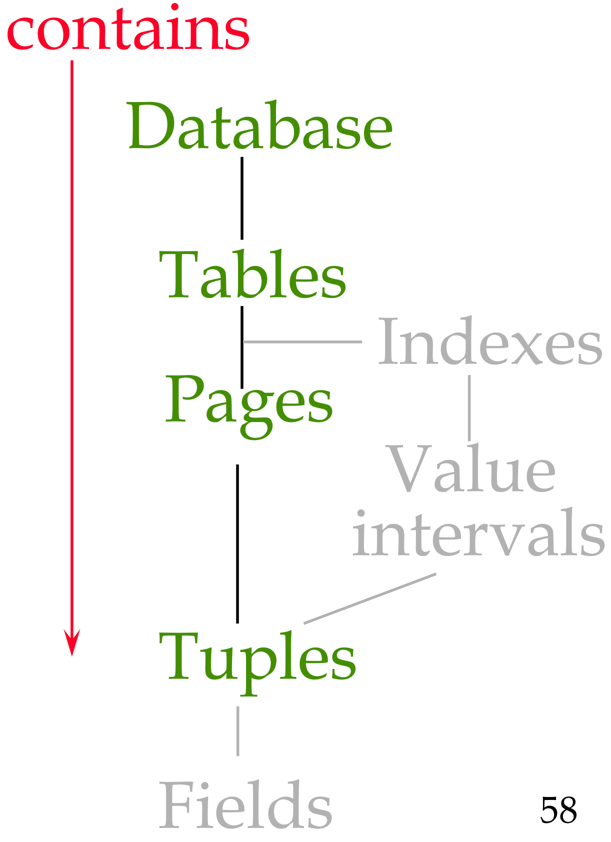


Multiple-Granularity Locks

- Granularity of locks: concurrency vs. overhead
  - Finer granularity increases concurrency
  - It, however, increases overhead in set/clear operations and storage of the lock(s)

- Hard to decide what granularity to lock (tables vs. pages vs. tuples).

- Key insight: shouldn't have to decide because data "containers" are nested:



contains

Database

Tables

Indexes

Pages

Value intervals

Tuples

Fields

New Lock Modes

- Each Xact sets "intention locks" (**IS** or **IX**), on all ancestors of an item that it will read or write.

- Compatibility:
  - IS, IX are compatible
  - IX, IX are also compatible
  - Increased concurrency!

T1\T2	--	IS	IX	S	X
--	√	√	√	√	√
IS	√	√	√	√	
IX	√	√	√		
S	√	√		√	
X	√				

- **SIX** mode: like S & IX at the same time.
  - SIX(R): an Xact reads the table R but only modifies a few tuples.
  - Not compatible with other IX.

Multiple Granularity Lock Protocol

- Each Xact starts from the root of the hierarchy.
- To get **S** or **IS** lock on a node, must hold **IS** or **IX** on parent.
- To get **X**, **IX** or **SIX** on a node, must hold **IX** or **SIX** on parent.
- Must release locks in bottom-up order.

The protocol is correct because it is equivalent to directly setting locks at the leaf levels of the hierarchy.

Examples

- T1 scans R, and updates a few tuples:
  - T1 gets an SIX lock on R, then repeatedly gets an S lock on tuples of R (which is implicit), and occasionally upgrades to X only on the right tuples.
- T2 uses an index to read only part of R:
  - T2 gets an IS lock on R, and repeatedly gets an S lock on tuples of R (should be explicit).
- T3 reads all of R:
  - T3 gets an S lock on R.

	--	IS	IX	S	X
--	√	√	√	√	√
IS	√	√	√	√	
IX	√	√	√		
S	√	√		√	
X	√				

Transaction Support in SQL-99

- Each transaction has an *access mode* (*Read Only* or not), and an *isolation level*.

Anomaly Isolation level	Dirty Read	Unrepeatable Read	Phantom Problem
1. *Read Uncommitted*	Maybe	Maybe	Maybe
2. *Read Committed*	No	Maybe	Maybe
3. *Repeatable Reads*	No	No	Maybe
4. *Serializable*	No	No	No

Strict 2PL
Index locking

SET TRANSACTION ISOLATION LEVEL *Serializable Read Only*

Summary

- There are several **lock-based concurrency control schemes** (Strict 2PL, 2PL). Conflicts between xacts can be detected in the dependency graph.
- The **lock manager** keeps track of the locks issued. **Deadlocks** can either be prevented or detected.
- **Multiple granularity locking** reduces the overhead involved in setting locks for nested collections of objects (e.g., a file of pages).

- PostgreSQL: https://www.postgresql.org/docs/current/transaction-iso.html

Summary (Contd.)

- SQL-99 provides different isolation levels that control the degree of concurrency.
- Optimistic CC aims to minimize CC overheads in an "optimistic" environment where reads are common and writes are rare.
- Optimistic CC has its own overhead however.
- **Timestamp CC** (not in the course) is another alternative to 2PL; allows some serializable schedules that 2PL does not (although converse is also true).
- Ensuring recoverability with Timestamp CC requires ability to block Xacts, similar to locking.